

Public EV Charging Access in New Jersey

Steps/Goals:

1. Load ACS, station, and tract boundary data.
2. Clean ACS values before and after cleaning.
3. Spatially assign charging stations to census tracts.
4. Build tract-level rates and EDA plots before mapping or regression.
5. Use maps and models

1. Load data

We load three inputs: ACS demographic variables, the NJ public EV charging station file, and NJ census tract boundaries. The tract `GEOID` is the common key that lets us join Census attributes and charging counts later.

In [510...

```
library(sf)
library(dplyr)
library(jsonlite)
library(ggplot2)

sf::sf_use_s2(FALSE)
Sys.setenv(SHAPE_RESTORE_SHX = "YES")
```

In [511...

```
demographic_data <- jsonlite::fromJSON("acs5_NJ.json")

ev_charging_stations <- read.csv("alt_fuel_stations_NJ.csv")

shp <- st_read("tl_2024_34_tract/tl_2024_34_tract.shp")
```

```

Reading layer `tl_2024_34_tract' from data source
`C:\Users\felin\Documents\R Folder\Spatial_Stats\EV_Charges\tl_2024_34_tract\tl_2024_34_tract.shp'
using driver `ESRI Shapefile'
Simple feature collection with 2181 features and 13 fields
Geometry type: MULTIPOLYGON
Dimension:      XY
Bounding box:   xmin: -75.56359 ymin: 38.78866 xmax: -73.88506 ymax: 41.35761
Geodetic CRS:   NAD83

```

2. Inspect inputs

verifyig the ACS columns, the tract shapefile fields, coordinate reference system, and station file dimensions before building

In [512...

```

demographic_data[1, ]
head(demographic_data[-1, ], 3)

```

```

'NAME' · 'B01003_001E' · 'B19013_001E' · 'B17001_001E' · 'B17001_002E' · 'B08201_001E' · 'B08201_002E' · 'B25003_001E' · 'B25003_002E' · 'state' ·
'county' · 'tract'

```

A matrix: 3 × 12 of type chr

Census Tract 1; Atlantic County; New Jersey	2380	56222	2360	478	813	175	813	386	34	001	000100
Census Tract 2; Atlantic County; New Jersey	3009	42346	3009	624	1457	328	1457	753	34	001	000200
Census Tract 3; Atlantic County; New Jersey	4064	52763	4064	1149	1474	606	1474	466	34	001	000300

In [513...

```

names(shp)
st_crs(shp)
dim(shp)

names(ev_charging_stations)
dim(ev_charging_stations)

```

```

'STATEFP' · 'COUNTYFP' · 'TRACTCE' · 'GEOID' · 'GEOIDFQ' · 'NAME' · 'NAMELSAD' · 'MTFCC' · 'FUNCSTAT' · 'ALAND' · 'AWATER' · 'INTPTLAT' ·
'INTPTLON' · 'geometry'

```

Coordinate Reference System:

User input: NAD83

wkt:

```
GEOGCRS["NAD83",  
  DATUM["North American Datum 1983",  
    ELLIPSOID["GRS 1980",6378137,298.257222101,  
      LENGTHUNIT["metre",1]],  
    PRIMEM["Greenwich",0,  
      ANGLEUNIT["degree",0.0174532925199433]],  
  CS[ellipsoidal,2],  
    AXIS["latitude",north,  
      ORDER[1],  
      ANGLEUNIT["degree",0.0174532925199433]],  
    AXIS["longitude",east,  
      ORDER[2],  
      ANGLEUNIT["degree",0.0174532925199433]],  
  ID["EPSG",4269]]
```

2181 · 14

'Fuel.Type.Code' · 'Station.Name' · 'Street.Address' · 'Intersection.Directions' · 'City' · 'State' · 'ZIP' · 'Plus4' · 'Station.Phone' · 'Status.Code' ·
'Expected.Date' · 'Groups.With.Access.Code' · 'Access.Days.Time' · 'Cards.Accepted' · 'BD.Blends' · 'NG.Fill.Type.Code' · 'NG.PSI' ·
'EV.Level1.EVSE.Num' · 'EV.Level2.EVSE.Num' · 'EV.DC.Fast.Count' · 'EV.Other.Info' · 'EV.Network' · 'EV.Network.Web' · 'Geocode.Status' · 'Latitude' ·
'Longitude' · 'Date.Last.Confirmed' · 'ID' · 'Updated.At' · 'Owner.Type.Code' · 'Federal.Agency.ID' · 'Federal.Agency.Name' · 'Open.Date' ·
'Hydrogen.Status.Link' · 'NG.Vehicle.Class' · 'LPG.Primary' · 'E85.Blender.Pump' · 'EV.Connector.Types' · 'Country' · 'Intersection.Directions..French.' ·
'Access.Days.Time..French.' · 'BD.Blends..French.' · 'Groups.With.Access.Code..French.' · 'Hydrogen.Is.Retail' · 'Access.Code' · 'Access.Detail.Code' ·
'Federal.Agency.Code' · 'Facility.Type' · 'CNG.Dispenser.Num' · 'CNG.On.Site.Renewable.Source' · 'CNG.Total.Compression.Capacity' ·
'CNG.Storage.Capacity' · 'LNG.On.Site.Renewable.Source' · 'E85.Other.Ethanol.Blends' · 'EV.Pricing' · 'EV.Pricing..French.' · 'LPG.Nozzle.Types' ·
'Hydrogen.Pressures' · 'Hydrogen.Standards' · 'CNG.Fill.Type.Code' · 'CNG.PSI' · 'CNG.Vehicle.Class' · 'LNG.Vehicle.Class' ·
'EV.On.Site.Renewable.Source' · 'Restricted.Access' · 'RD.Blends' · 'RD.Blends..French.' · 'RD.Blended.with.Biodiesel' · 'RD.Maximum.Biodiesel.Level' ·
'NPS.Unit.Name' · 'CNG.Station.Sells.Renewable.Natural.Gas' · 'LNG.Station.Sells.Renewable.Natural.Gas' · 'Maximum.Vehicle.Class' ·
'EV.Workplace.Charging' · 'Funding.Sources'

1822 · 75

3. Clean ACS variables

ACS API values arrive as text, so need we convert the fields we need into numeric variables. We also handle two common problems:

- Census missing-value sentinels, such as very large negative income values, are converted to `NA`.
- Percent/rate variables are only calculated when their denominator is positive.

```
In [ ]: # Convertfig the Census API array into a data frame and give it the header row as column names.
acs_df <- as.data.frame(demographic_data[-1, ])
names(acs_df) <- demographic_data[1, ]

# Keep raw before and after cleaning.
acs_raw <- acs_df %>%
  mutate(
    GEOID = paste0(state, county, tract),
    total_population = as.numeric(B01003_001E),
    median_income_raw = as.numeric(B19013_001E),
    poverty_total = as.numeric(B17001_001E),
    poverty_below = as.numeric(B17001_002E),
    households_total = as.numeric(B08201_001E),
    households_no_vehicle = as.numeric(B08201_002E),
    housing_total = as.numeric(B25003_001E),
    owner_occupied = as.numeric(B25003_002E)
  )

# Census uses large negative values as missing-value sentinels for some estimates.
# Rates are only calculated when their denominators are positive.
acs_clean <- acs_raw %>%
  mutate(
    median_income = ifelse(median_income_raw < 0, NA_real_, median_income_raw),
    poverty_rate = ifelse(poverty_total > 0, 100 * poverty_below / poverty_total, NA_real_),
    pct_no_vehicle = ifelse(households_total > 0, 100 * households_no_vehicle / households_total, NA_real_),
    pct_owner_occupied = ifelse(housing_total > 0, 100 * owner_occupied / housing_total, NA_real_)
  )

# This table makes the cleaning step visible
acs_cleaning_check <- tibble::tibble(
  check = c(
    "tract rows",
    "negative median income values",
    "zero poverty denominators",
```

```

    "zero household denominators",
    "zero housing denominators"
  ),
  before = c(
    nrow(acs_raw),
    sum(acs_raw$median_income_raw < 0, na.rm = TRUE),
    sum(acs_raw$poverty_total == 0, na.rm = TRUE),
    sum(acs_raw$households_total == 0, na.rm = TRUE),
    sum(acs_raw$housing_total == 0, na.rm = TRUE)
  ),
  after_missing = c(
    sum(is.na(acs_clean$GEOID)),
    sum(is.na(acs_clean$median_income)),
    sum(is.na(acs_clean$poverty_rate)),
    sum(is.na(acs_clean$pct_no_vehicle)),
    sum(is.na(acs_clean$pct_owner_occupied))
  )
)
acs_cleaning_check

```

A tibble: 5 × 3

	check	before	after_missing
	<chr>	<int>	<int>
	tract rows	2181	0
	negative median income values	31	31
	zero poverty denominators	13	13
	zero household denominators	15	15
	zero housing denominators	15	15

Spatial Join GEOID

In [515...]

```

tracts_acs <- shp %>%
  left_join(acs_clean, by = "GEOID")

```

```
sum(is.na(tracts_acs$total_population))
```

0

Filters the station file to usable public, open NJ charging stations and calculates Level 2, DC fast, and total port counts.

```
In [516... ev_clean <- ev_charging_stations %>%
  filter(
    State == "NJ",
    Access.Code == "public",
    Status.Code == "E",
    !is.na(Latitude),
    !is.na(Longitude)
  ) %>%
  mutate(
    level2_ports = ifelse(is.na(EV.Level2.EVSE.Num), 0, EV.Level2.EVSE.Num),
    dc_fast_ports = ifelse(is.na(EV.DC.Fast.Count), 0, EV.DC.Fast.Count),
    ports_total = level2_ports + dc_fast_ports
  )

nrow(ev_clean)
summary(ev_clean$level2_ports)
summary(ev_clean$dc_fast_ports)
```

1822

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
0.000	1.000	2.000	2.176	2.000	49.000
Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
0.000	0.000	0.000	1.132	0.000	40.000

Convert station longitude/latitude coordinates into spatial points.

```
In [517... ev_sf <- st_as_sf(
  ev_clean,
  coords = c("Longitude", "Latitude"),
  crs = 4326,
  remove = FALSE
)
```

```
ev_sf <- st_transform(ev_sf, st_crs(tracts_acs))
```

4. Assign stations to tracts

Charging stations are point locations, while ACS data are tract polygons. We transform both layers to EPSG 5070 before the spatial join so spatial operations run in a projected coordinate system. Then we count stations and ports inside each tract.

Projects tracts and stations to EPSG 5070 and assigns each station to the tract containing it.

```
In [ ]: # EPSG 5070 is an equal-area projected CRS commonly used for US spatial analysis.
tracts_acs_proj <- st_transform(tracts_acs, 5070)
ev_sf_proj <- st_transform(ev_sf, 5070)

# Assign each station point to the tract polygon that contains it.
chargers_joined <- st_join(
  ev_sf_proj,
  tracts_acs_proj,
  join = st_within
)

# A result of 0 means every station was assigned to a tract.
sum(is.na(chargers_joined$GEOID))
```

0

Check if any stations failed to join to a census tract.

```
In [519... chargers_joined %>%
  st_drop_geometry() %>%
  summarise(
    stations_joined = n(),
    stations_missing_tract = sum(is.na(GEOID)),
    pct_missing_tract = mean(is.na(GEOID)) * 100
  )
```

A data.frame: 1 × 3

stations_joined	stations_missing_tract	pct_missing_tract
<int>	<int>	<dbl>
1822	0	0

Aggregate station and port totals by tract.

```
In [520... charger_counts <- chargers_joined %>%  
  st_drop_geometry() %>%  
  filter(!is.na(GEOID)) %>%  
  group_by(GEOID) %>%  
  summarise(  
    stations_total = n(),  
    level2_ports = sum(level2_ports, na.rm = TRUE),  
    dc_fast_ports = sum(dc_fast_ports, na.rm = TRUE),  
    ports_total = sum(ports_total, na.rm = TRUE),  
    .groups = "drop"  
  )
```

5. Build tract-level analysis data

After counting stations by tract, we join those counts back to every tract. Tracts without a matched station are assigned zero stations and zero ports. We then calculate per-capita rates.

We keep three versions of the station rate:

- `stations_per_10k_raw` : the direct population-adjusted rate.
- `stations_per_10k_clean` : excludes tracts with population below 500 because tiny denominators can create misleadingly huge rates.
- `stations_per_10k_map` : caps the cleaned rate at 75 only for map display.

```
In [ ]: # Join tract-level charger counts back to the tract polygons.  
# Missing counts are true zeros: those tracts have no public charging stations in the station file.  
final_data <- tracts_acs %>%  
  left_join(charger_counts, by = "GEOID") %>%
```



```

mutate(
  stations_total = ifelse(is.na(stations_total), 0, stations_total),
  level2_ports = ifelse(is.na(level2_ports), 0, level2_ports),
  dc_fast_ports = ifelse(is.na(dc_fast_ports), 0, dc_fast_ports),
  ports_total = ifelse(is.na(ports_total), 0, ports_total),

  # Raw rates
  stations_per_10k_raw = ifelse(
    total_population > 0,
    10000 * stations_total / total_population,
    NA_real_
  ),
  # Clean rates exclude very low-population tracts before mapping and modeling.
  stations_per_10k_clean = ifelse(
    total_population >= 500,
    stations_per_10k_raw,
    NA_real_
  ),
  # The capped version is only for display prevents a few outliers from flattening the map scale.
  stations_per_10k_map = pmin(stations_per_10k_clean, 75),

  ports_per_10k_raw = ifelse(
    total_population > 0,
    10000 * ports_total / total_population,
    NA_real_
  ),
  ports_per_10k_clean = ifelse(
    total_population >= 500,
    ports_per_10k_raw,
    NA_real_
  ),
  has_station = stations_total > 0
)

```

6. Exploratory data analysis

EDA comes before maps, spatial statistics, and regression. starting with broad summaries, then look at geographic totals by county, then examine tract-level distributions and simple relationships.

This order matters because it helps us catch skew, outliers, zero-heavy outcomes, and denominator problems before choosing statistical methods.

In [522...

```
eda_summary <- final_data %>%
  st_drop_geometry() %>%
  summarise(
    total_tracts = n(),
    total_stations = sum(stations_total, na.rm = TRUE),
    total_level2_ports = sum(level2_ports, na.rm = TRUE),
    total_dc_fast_ports = sum(dc_fast_ports, na.rm = TRUE),
    total_ports = sum(ports_total, na.rm = TRUE),
    tracts_with_station = sum(stations_total > 0, na.rm = TRUE),
    tracts_without_station = sum(stations_total == 0, na.rm = TRUE),
    pct_tracts_with_station = mean(stations_total > 0, na.rm = TRUE) * 100,
    median_stations_per_10k = median(stations_per_10k_clean, na.rm = TRUE),
    mean_stations_per_10k = mean(stations_per_10k_clean, na.rm = TRUE)
  )

eda_summary
```

A data.frame: 1 × 10

total_tracts	total_stations	total_level2_ports	total_dc_fast_ports	total_ports	tracts_with_station	tracts_without_station	pct_tracts_with_station
<int>	<dbl>	<dbl>	<dbl>	<dbl>	<int>	<int>	<dbl>
2181	1822	3965	2062	6027	701	1480	32.1412

keeping tracts with population at least 500 and reports core summary statistics.

In [523...

```
eda_df <- final_data %>%
  st_drop_geometry() %>%
  filter(total_population >= 500)

# Plot-specific datasets make missing-value exclusions explicit.
eda_rate_df <- eda_df %>%
  filter(is.finite(stations_per_10k_clean))

eda_income_box_df <- eda_df %>%
```

```

filter(!is.na(has_station), is.finite(median_income))

eda_poverty_df <- eda_df %>%
  filter(is.finite(poverty_rate), is.finite(stations_per_10k_clean))

eda_no_vehicle_df <- eda_df %>%
  filter(is.finite(pct_no_vehicle), is.finite(stations_per_10k_clean))

eda_df %>%
  summarise(
    tracts = n(),
    tracts_with_station = sum(has_station),
    pct_tracts_with_station = mean(has_station) * 100,
    median_population = median(total_population, na.rm = TRUE),
    median_income = median(median_income, na.rm = TRUE),
    median_stations_per_10k = median(stations_per_10k_clean, na.rm = TRUE),
    mean_stations_per_10k = mean(stations_per_10k_clean, na.rm = TRUE),
    max_stations_per_10k = max(stations_per_10k_clean, na.rm = TRUE)
  )

```

A data.frame: 1 × 8

tracts	tracts_with_station	pct_tracts_with_station	median_population	median_income	median_stations_per_10k	mean_stations_per_10k	max
<int>	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	
2165	698	32.24018	4178	103487	0	2.05281	

Showing how many rows are used or removed for each EDA plot.

In [524...

```

eda_plot_missing_check <- tibble::tibble(
  plot = c(
    "stations per 10k histogram",
    "income boxplot",
    "poverty scatterplot",
    "no-vehicle scatterplot"
  ),
  rows_available = nrow(eda_df),
  rows_used = c(
    nrow(eda_rate_df),

```

```

    nrow(eda_income_box_df),
    nrow(eda_poverty_df),
    nrow(eda_no_vehicle_df)
  ),
  rows_removed = rows_available - rows_used
)

eda_plot_missing_check

```

A tibble: 4 × 4

plot	rows_available	rows_used	rows_removed
<chr>	<int>	<int>	<int>
stations per 10k histogram	2165	2165	0
income boxplot	2165	2149	16
poverty scatterplot	2165	2161	4
no-vehicle scatterplot	2165	2160	5

Adding county names and summarizes charging stations by county.

```

In [525... nj_counties <- data.frame(
  COUNTYFP = c(
    "001", "003", "005", "007", "009", "011", "013",
    "015", "017", "019", "021", "023", "025", "027",
    "029", "031", "033", "035", "037", "039", "041"
  ),
  county_name = c(
    "Atlantic", "Bergen", "Burlington", "Camden", "Cape May",
    "Cumberland", "Essex", "Gloucester", "Hudson", "Hunterdon",
    "Mercer", "Middlesex", "Monmouth", "Morris", "Ocean",
    "Passaic", "Salem", "Somerset", "Sussex", "Union", "Warren"
  )
)

county_summary <- final_data %>%
  st_drop_geometry() %>%

```

```
group_by(COUNTYFP) %>%
summarise(
  total_stations = sum(stations_total, na.rm = TRUE),
  tracts = n(),
  tracts_with_station = sum(stations_total > 0, na.rm = TRUE),
  pct_tracts_with_station = mean(stations_total > 0, na.rm = TRUE) * 100,
  .groups = "drop"
) %>%
left_join(nj_counties, by = "COUNTYFP") %>%
arrange(desc(total_stations))

county_summary
```

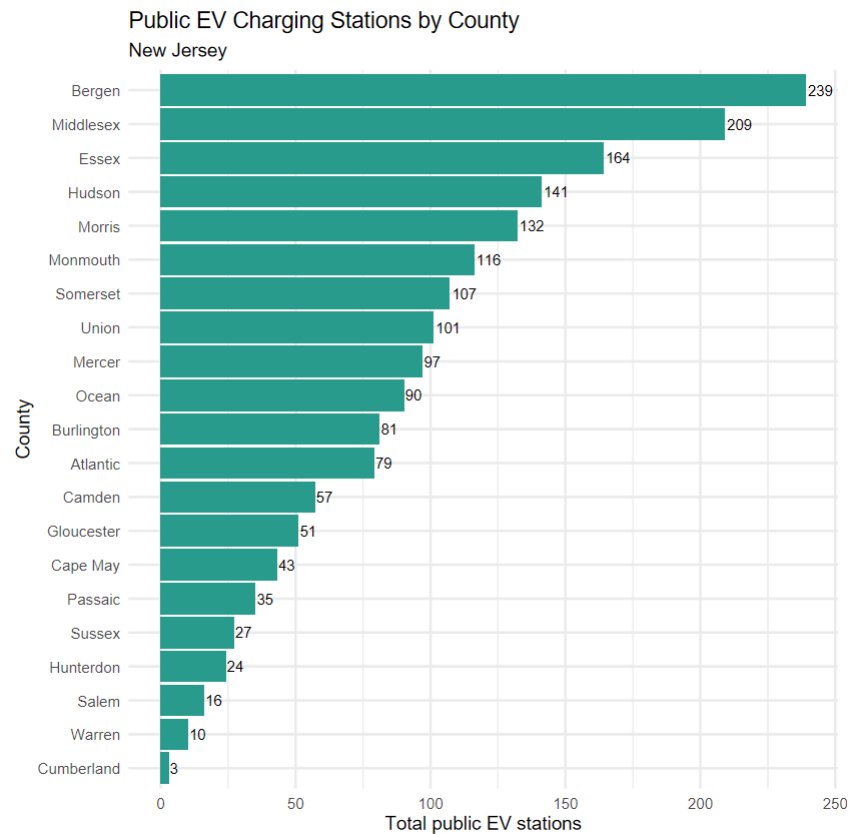
A tibble: 21 × 6

COUNTYFP	total_stations	tracts	tracts_with_station	pct_tracts_with_station	county_name
<chr>	<dbl>	<int>	<int>	<dbl>	<chr>
003	239	203	74	36.453202	Bergen
023	209	192	79	41.145833	Middlesex
013	164	211	55	26.066351	Essex
017	141	183	56	30.601093	Hudson
027	132	110	59	53.636364	Morris
025	116	155	46	29.677419	Monmouth
035	107	74	37	50.000000	Somerset
039	101	120	37	30.833333	Union
021	97	84	36	42.857143	Mercer
029	90	145	38	26.206897	Ocean
005	81	117	39	33.333333	Burlington
001	79	74	24	32.432432	Atlantic
007	57	129	28	21.705426	Camden
015	51	69	25	36.231884	Gloucester
009	43	33	16	48.484848	Cape May
031	35	120	11	9.166667	Passaic
037	27	42	14	33.333333	Sussex
019	24	30	11	36.666667	Hunterdon
033	16	25	6	24.000000	Salem
041	10	23	7	30.434783	Warren

COUNTYFP	total_stations	tracts	tracts_with_station	pct_tracts_with_station	county_name
<chr>	<dbl>	<int>	<int>	<dbl>	<chr>
011	3	42	3	7.142857	Cumberland

Total public charging stations by county.

```
In [526... ggplot(county_summary, aes(x = reorder(county_name, total_stations), y = total_stations)) +
  geom_col(fill = "#2A9D8F") +
  geom_text(
    aes(label = total_stations),
    hjust = -0.1,
    size = 3
  ) +
  coord_flip() +
  labs(
    title = "Public EV Charging Stations by County",
    subtitle = "New Jersey",
    x = "County",
    y = "Total public EV stations"
  ) +
  theme_minimal()
```



Comparing tracts with at least one station to tracts with no station.

```
In [527... eda_group_comparison <- eda_df %>%
  group_by(has_station) %>%
  summarise(
    tracts = n(),
    median_population = median(total_population, na.rm = TRUE),
    median_income = median(median_income, na.rm = TRUE),
    median_poverty_rate = median(poverty_rate, na.rm = TRUE),
    median_pct_no_vehicle = median(pct_no_vehicle, na.rm = TRUE),
    median_pct_owner_occupied = median(pct_owner_occupied, na.rm = TRUE),
    median_ports_total = median(ports_total, na.rm = TRUE),
    .groups = "drop"
  )
```



```
eda_group_comparison
```

A tibble: 2 × 8

has_station	tracts	median_population	median_income	median_poverty_rate	median_pct_no_vehicle	median_pct_owner_occupied	median_p
<lgl>	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
FALSE	1467	4019	98265	7.61564	6.382979	69.36236	
TRUE	698	4589	112529	6.06062	5.980529	70.40582	

Compute simple correlations between station rate and selected tract characteristics.

```
In [528... eda_correlations <- eda_df %>%  
  summarise(  
    corr_income = cor(stations_per_10k_clean, median_income, use = "complete.obs"),  
    corr_poverty = cor(stations_per_10k_clean, poverty_rate, use = "complete.obs"),  
    corr_no_vehicle = cor(stations_per_10k_clean, pct_no_vehicle, use = "complete.obs"),  
    corr_owner_occupied = cor(stations_per_10k_clean, pct_owner_occupied, use = "complete.obs"),  
    corr_population = cor(stations_per_10k_clean, total_population, use = "complete.obs")  
  )  
  
eda_correlations
```

A data.frame: 1 × 5

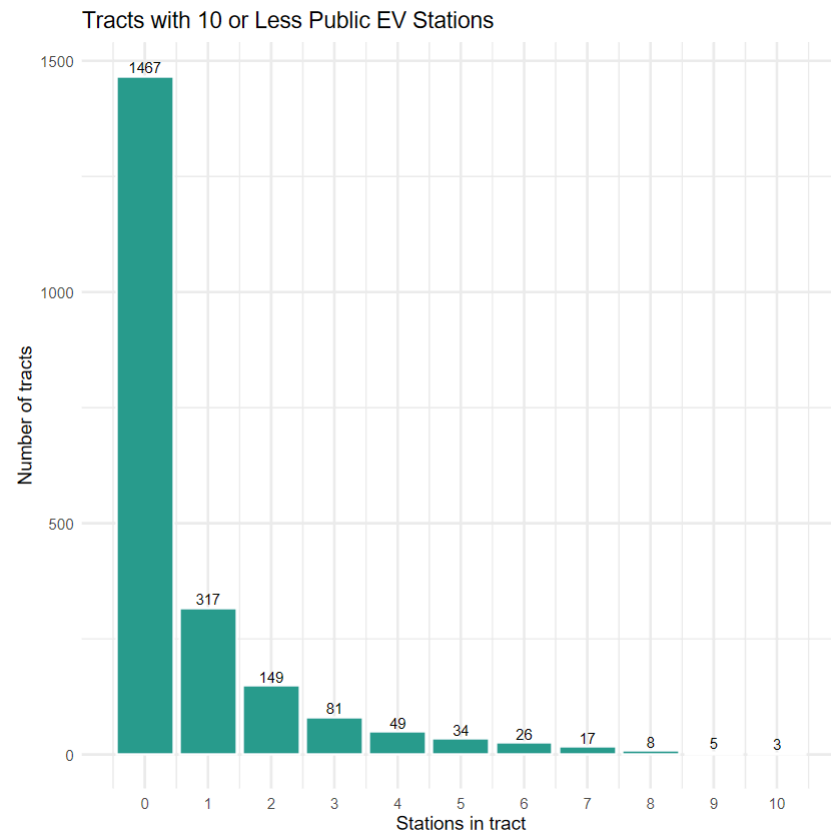
corr_income	corr_poverty	corr_no_vehicle	corr_owner_occupied	corr_population
<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
0.02552796	0.008949666	0.0374008	-0.0452497	-0.07333558

distribution of station counts per tract.

```
In [529... station_hist_data <- eda_df %>%  
  st_drop_geometry() %>%  
  count(stations_total, name = "n_tracts")
```

```
plot_low <- ggplot(
  station_hist_data %>% filter(stations_total <= 10),
  aes(x = stations_total, y = n_tracts)
) +
  geom_col(fill = "#2A9D8F", color = "white") +
  geom_text(
    aes(label = n_tracts),
    vjust = -0.25,
    size = 3
  ) +
  scale_x_continuous(breaks = 0:10) +
  labs(
    title = "Tracts with 10 or Less Public EV Stations",
    x = "Stations in tract",
    y = "Number of tracts"
  ) +
  theme_minimal()

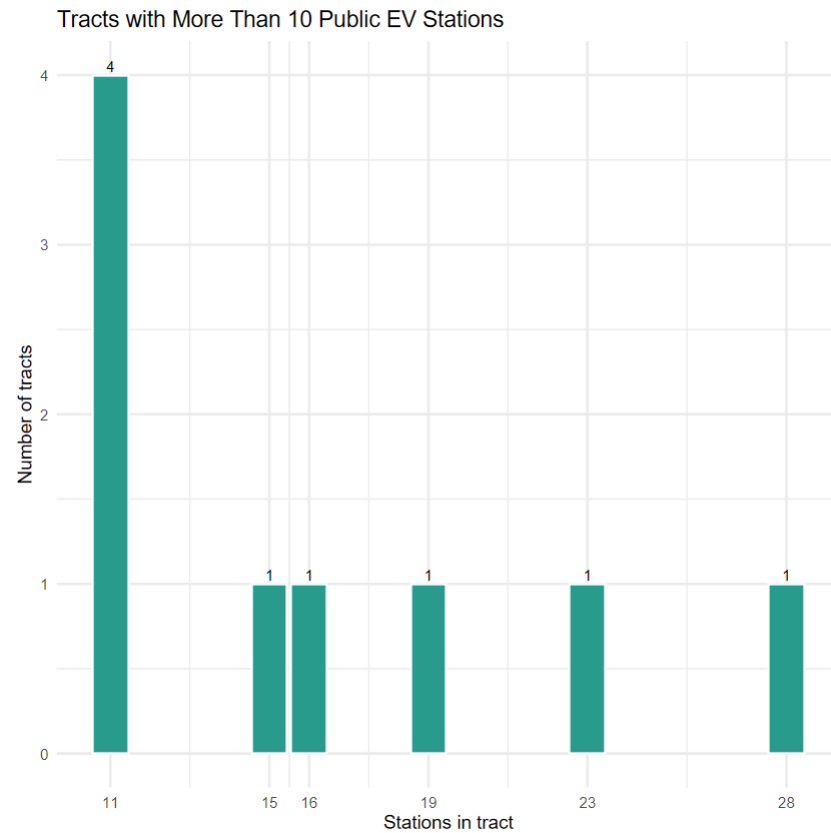
plot_low
```



```
In [530... plot_high <- ggplot(  
  station_hist_data %>% filter(stations_total > 10),  
  aes(x = stations_total, y = n_tracts)  
) +  
  geom_col(fill = "#2A9D8F", color = "white") +  
  geom_text(  
    aes(label = n_tracts),  
    vjust = -0.25,  
    size = 3  
  ) +  
  scale_x_continuous(  
    breaks = sort(unique(station_hist_data$stations_total[station_hist_data$stations_total > 10]))  
  ) +  
  labs(  
    title = "Tracts with More Than 10 Public EV Stations",
```

```
x = "Stations in tract",
y = "Number of tracts"
) +
theme_minimal()
```

plot_high



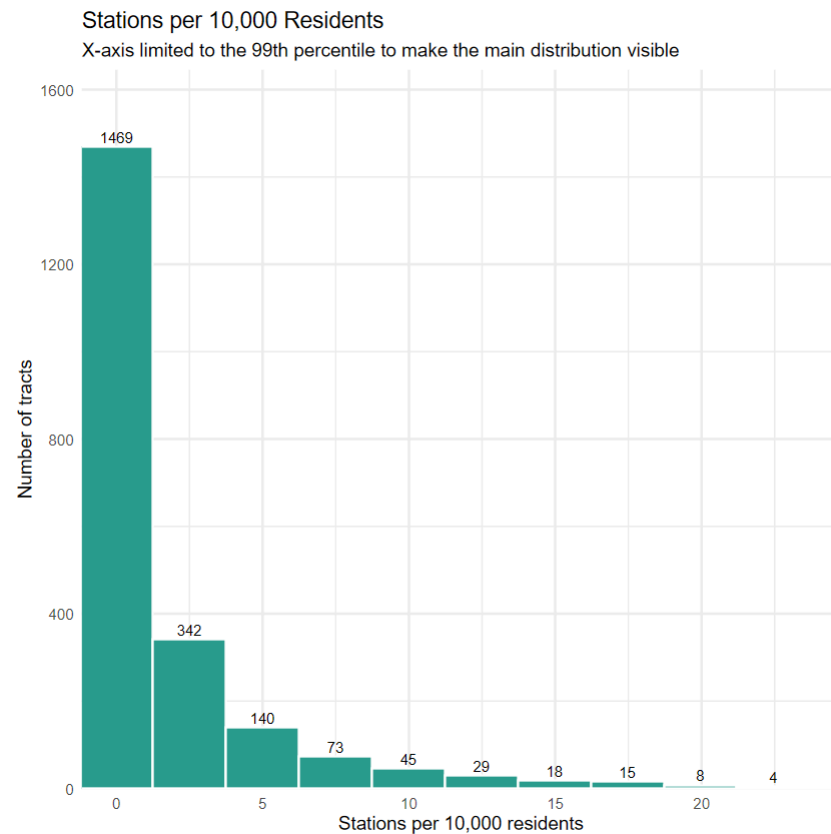
distribution of cleaned stations per 10,000 residents.

```
In [531... ggplot(eda_rate_df, aes(x = stations_per_10k_clean)) +
  geom_histogram(bins = 40, fill = "#2A9D8F", color = "white") +
  geom_text(
    stat = "bin",
    bins = 40,
    aes(
```

```

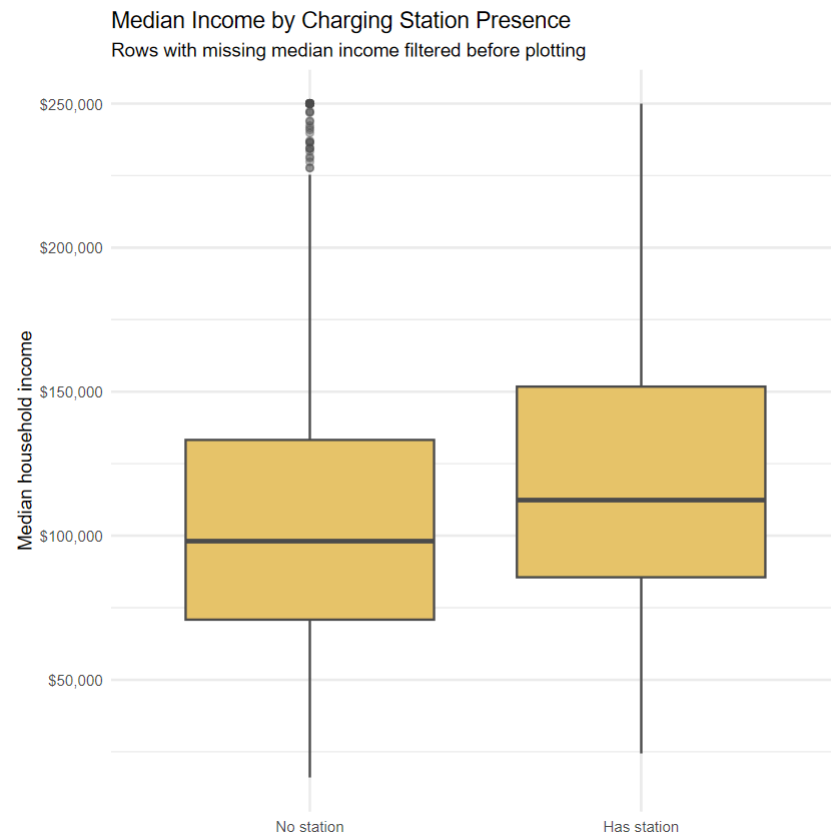
    y = after_stat(count),
    label = after_stat(ifelse(count > 0, count, ""))
  ),
  vjust = -0.3,
  size = 3
) +
coord_cartesian(
  xlim = c(0, quantile(eda_rate_df$stations_per_10k_clean, 0.99, na.rm = TRUE))
) +
scale_y_continuous(expand = expansion(mult = c(0, 0.12))) +
labs(
  title = "Stations per 10,000 Residents",
  subtitle = "X-axis limited to the 99th percentile to make the main distribution visible",
  x = "Stations per 10,000 residents",
  y = "Number of tracts"
) +
theme_minimal()

```



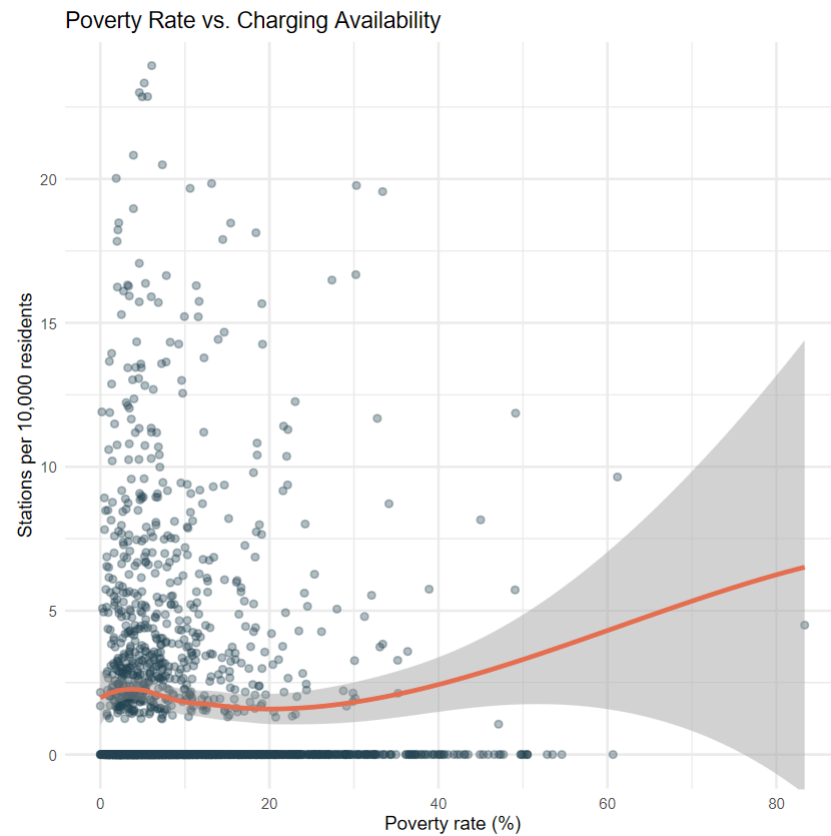
Median household income for tracts with and without stations

```
In [532... ggplot(eda_income_box_df, aes(x = has_station, y = median_income)) +  
  geom_boxplot(fill = "#E9C46A", color = "gray30", outlier.alpha = 0.35) +  
  scale_x_discrete(labels = c("No station", "Has station")) +  
  scale_y_continuous(labels = scales::dollar) +  
  labs(  
    title = "Median Income by Charging Station Presence",  
    subtitle = "Rows with missing median income filtered before plotting",  
    x = NULL,  
    y = "Median household income"  
  ) +  
  theme_minimal()
```



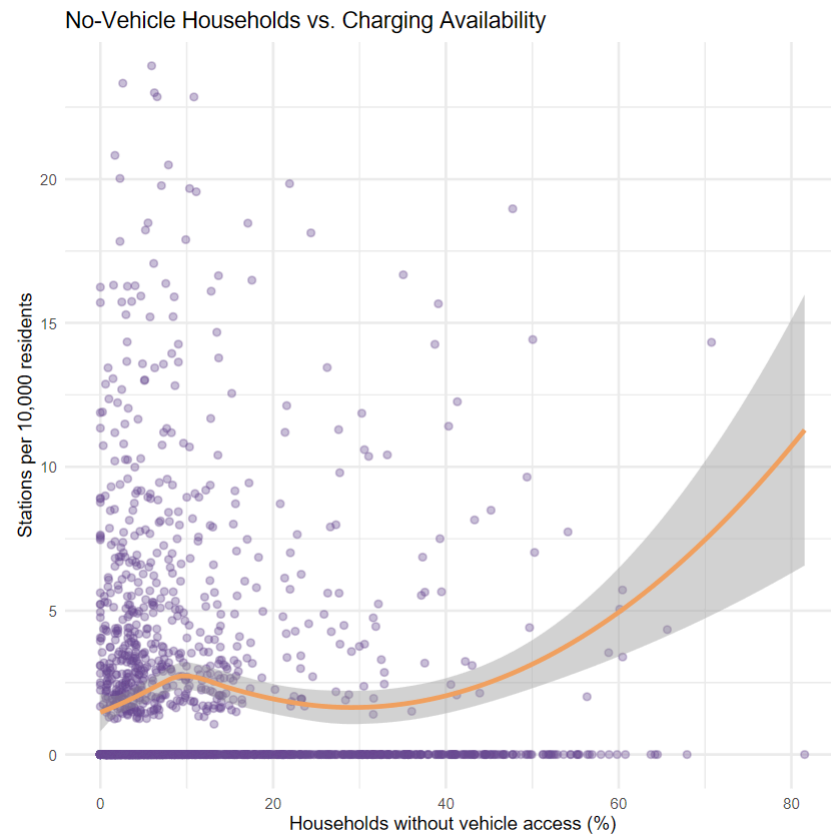
The relationship between poverty rate and charging availability.

```
In [533... ggplot(eda_poverty_df, aes(x = poverty_rate, y = stations_per_10k_clean)) +
  geom_point(alpha = 0.35, color = "#264653") +
  geom_smooth(method = "loess", formula = y ~ x, se = TRUE, color = "#E76F51") +
  coord_cartesian(y = c(0, quantile(eda_poverty_df$stations_per_10k_clean, 0.99, na.rm = TRUE))) +
  labs(
    title = "Poverty Rate vs. Charging Availability",
    x = "Poverty rate (%)",
    y = "Stations per 10,000 residents"
  ) +
  theme_minimal()
```



The relationship between no-vehicle households and charging availability.

```
In [534... ggplot(eda_no_vehicle_df, aes(x = pct_no_vehicle, y = stations_per_10k_clean)) +
  geom_point(alpha = 0.35, color = "#6A4C93") +
  geom_smooth(method = "loess", formula = y ~ x, se = TRUE, color = "#F4A261") +
  coord_cartesian(y = c(0, quantile(eda_no_vehicle_df$stations_per_10k_clean, 0.99, na.rm = TRUE))) +
  labs(
    title = "No-Vehicle Households vs. Charging Availability",
    x = "Households without vehicle access (%)",
    y = "Stations per 10,000 residents"
  ) +
  theme_minimal()
```

7. Highest-rate tracts

After the general EDA, we inspect the highest-rate tracts directly. This is an outlier check: high per-capita station rates can reflect destination charging areas, parking facilities, workplaces, shopping centers, or low residential population relative to station count.

```
In [535... final_data %>%  
  st_drop_geometry() %>%  
  arrange(desc(stations_per_10k_clean)) %>%  
  dplyr::select(  
    GEOID,  
    total_population,  
    stations_total,  
    level2_ports,
```

```

dc_fast_ports,
stations_per_10k_clean
) %>%
head(15)

```

A data.frame: 15 × 6

	GEOID	total_population	stations_total	level2_ports	dc_fast_ports	stations_per_10k_clean
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	34001002400	2364	23	24	4	97.29272
2	34003032103	4523	28	57	14	61.90581
3	34013006400	1373	7	16	1	50.98325
4	34023001417	2069	10	10	0	48.33253
5	34029738002	1223	5	9	0	40.88307
6	34033022201	1714	7	14	0	40.84014
7	34003023100	4674	19	39	14	40.65041
8	34003042500	4269	16	26	31	37.47950
9	34029728003	1734	6	12	0	34.60208
10	34025803601	2027	7	10	0	34.53379
11	34027043200	4532	15	16	0	33.09797
12	34009021500	1967	6	4	4	30.50330
13	34001001900	1681	5	10	0	29.74420
14	34025808501	3724	11	23	4	29.53813
15	34009022000	2767	8	13	2	28.91218

8. Mapping before and after cleaning

Only after the tabular EDA do we map the rate. The first map shows the raw rate, the cleaning table reports what changes, and the second map shows the cleaned/capped version used for visual interpretation.

The cap changes the map scale only; it does not change the underlying analysis variables used in summaries or modeling.

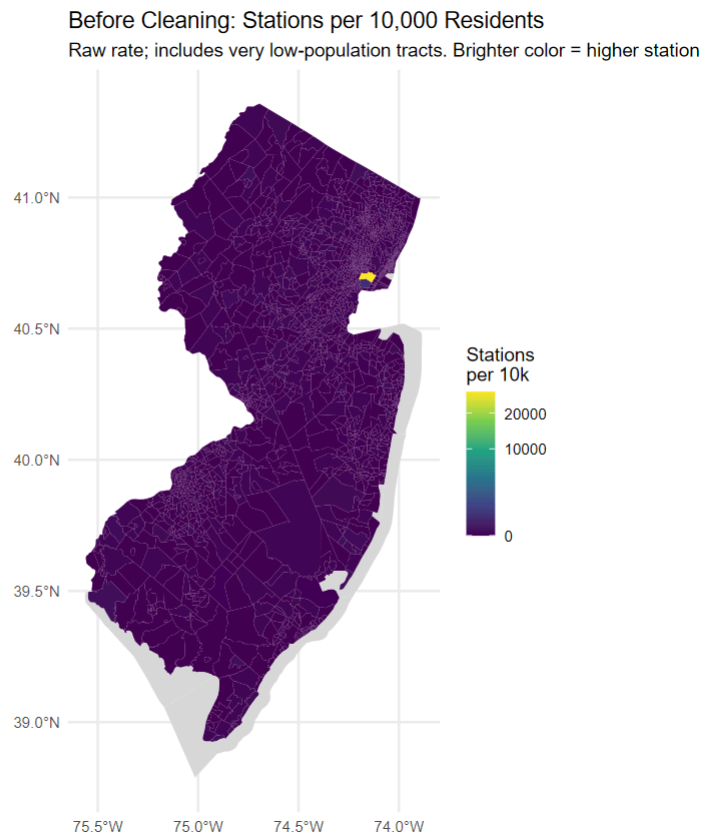
Map Color Choices

The maps use different color logic depending on the variable type:

- Charging availability and income use sequential scales where brighter color means a higher value.
- No-vehicle share uses a sequential scale because higher values identify stronger equity context.
- Distance to nearest charger uses a blue sequential scale where darker blue means farther away, which reads as lower access.
- Model residuals use a diverging blue-white-red scale centered at zero because residuals have meaningful negative and positive directions.

In [536...

```
ggplot(final_data) +  
  geom_sf(aes(fill = stations_per_10k_raw), color = NA) +  
  scale_fill_viridis_c(  
    option = "D",  
    trans = "sqrt",  
    na.value = "gray85",  
    name = "Stations\nper 10k"  
  ) +  
  labs(  
    title = "Before Cleaning: Stations per 10,000 Residents",  
    subtitle = "Raw rate; includes very low-population tracts. Brighter color = higher station availability."  
  ) +  
  theme_minimal()
```



```
In [537... map_cleaning_check <- final_data %>%  
  st_drop_geometry() %>%  
  summarise(  
    total_tracts = n(),  
    low_population_tracts_excluded = sum(total_population < 500, na.rm = TRUE),  
    raw_missing_rates = sum(is.na(stations_per_10k_raw)),  
    clean_missing_rates = sum(is.na(stations_per_10k_clean)),  
    values_capped_at_75 = sum(stations_per_10k_clean > 75, na.rm = TRUE),  
    max_raw_rate = max(stations_per_10k_raw, na.rm = TRUE),  
    max_clean_rate = max(stations_per_10k_clean, na.rm = TRUE),  
    max_map_rate = max(stations_per_10k_map, na.rm = TRUE)  
  )  
  
map_cleaning_check
```

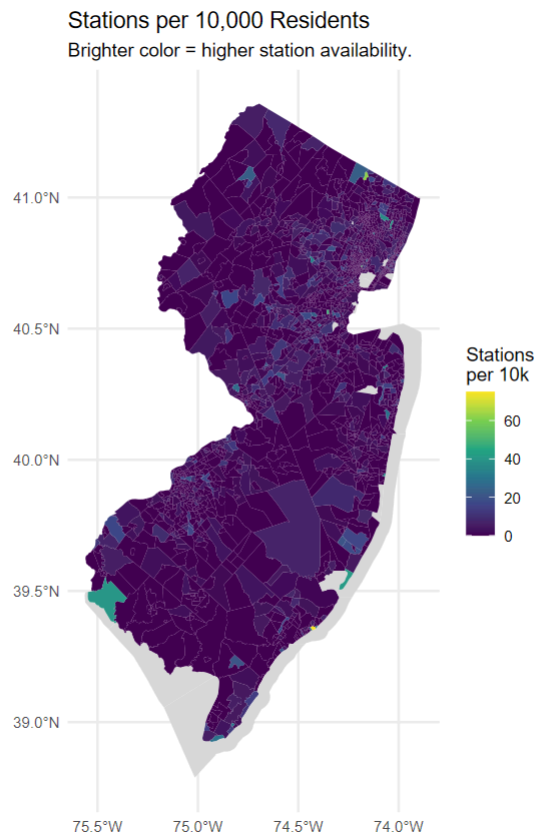
A data.frame: 1 × 8

total_tracts	low_population_tracts_excluded	raw_missing_rates	clean_missing_rates	values_capped_at_75	max_raw_rate	max_clean_rate	max
<int>	<int>	<int>	<int>	<int>	<dbl>	<dbl>	
2181	16	9	16	1	27500	97.29272	



In [538...

```
ggplot(final_data) +  
  geom_sf(aes(fill = stations_per_10k_map), color = NA) +  
  scale_fill_viridis_c(  
    option = "D",  
    na.value = "gray85",  
    name = "Stations\nper 10k"  
  ) +  
  labs(  
    title = "Stations per 10,000 Residents",  
    subtitle = "Brighter color = higher station availability."  
  ) +  
  theme_minimal()
```



9. Context and accessibility maps

These maps add geographic context after the main rate map. Income, no-vehicle households, station points, and distance to the nearest charger help us see whether charging availability appears concentrated in particular socioeconomic or spatial patterns.

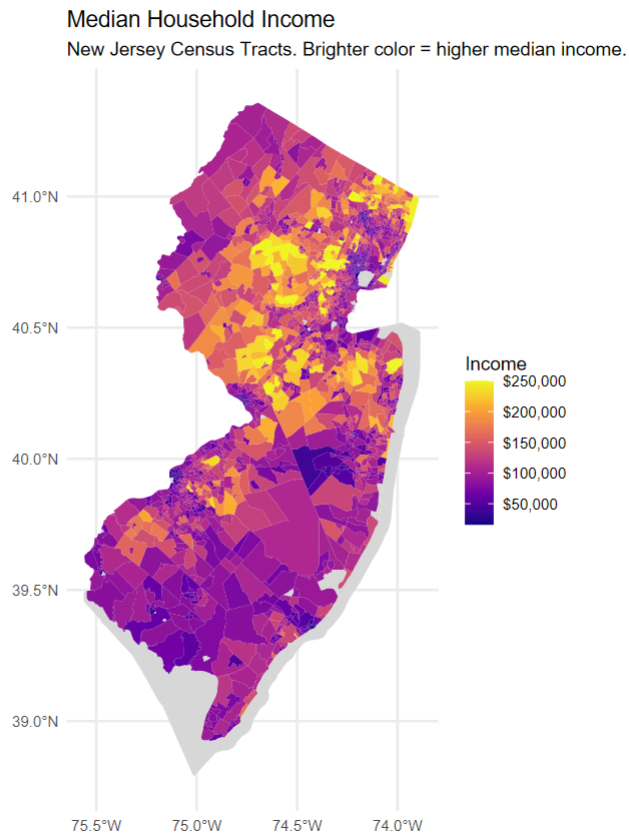
What this map shows: Median household income by tract. **Color interpretation:** Brighter color means higher income

```
In [539... ggplot(final_data) +  
  geom_sf(aes(fill = median_income), color = NA) +  
  scale_fill_viridis_c(  
    option = "C",  
    na.value = "gray85",
```

```

  labels = scales::dollar,
  name = "Income"
) +
labs(
  title = "Median Household Income",
  subtitle = "New Jersey Census Tracts. Brighter color = higher median income."
) +
theme_minimal()

```



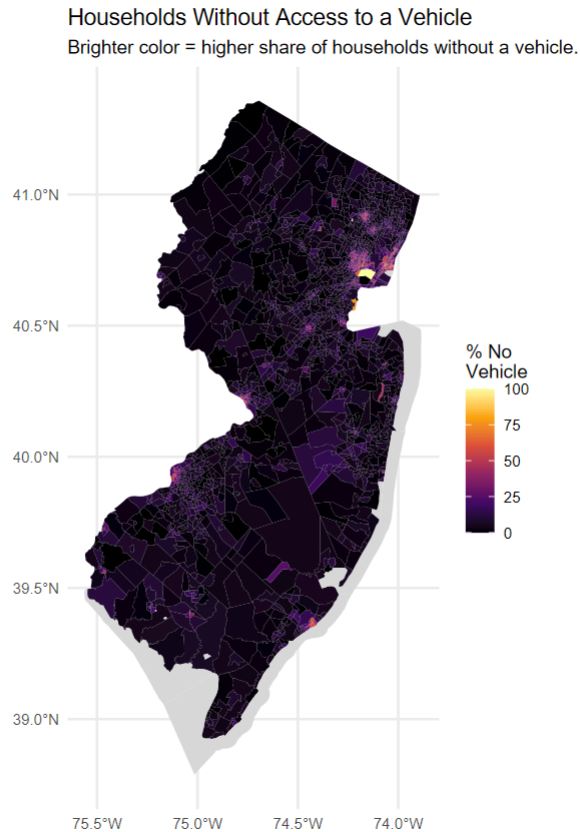
In [540...

```

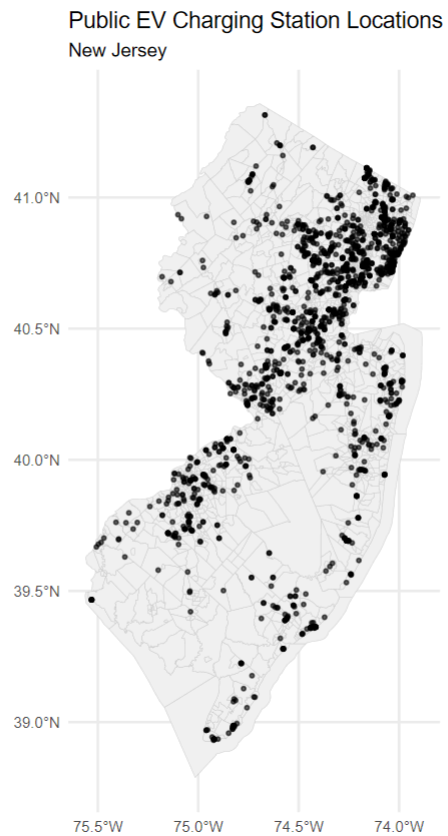
ggplot(final_data) +
  geom_sf(aes(fill = pct_no_vehicle), color = NA) +
  scale_fill_viridis_c(
    option = "B",
    na.value = "gray85",
    name = "% No\nVehicle"
  )

```

```
) +
labs(
  title = "Households Without Access to a Vehicle",
  subtitle = "Brighter color = higher share of households without a vehicle."
) +
theme_minimal()
```



```
In [541... ggplot() +
  geom_sf(data = final_data, fill = "gray95", color = "gray80", linewidth = 0.1) +
  geom_sf(data = ev_sf, size = 0.6, alpha = 0.6) +
  labs(
    title = "Public EV Charging Station Locations",
    subtitle = "New Jersey"
  ) +
  theme_minimal()
```

Distance to nearest charger

```
In [ ]: tract_geoms_5070 <- st_geometry(st_transform(final_data, 5070))
ev_geoms_5070 <- st_geometry(st_transform(ev_sf, 5070))

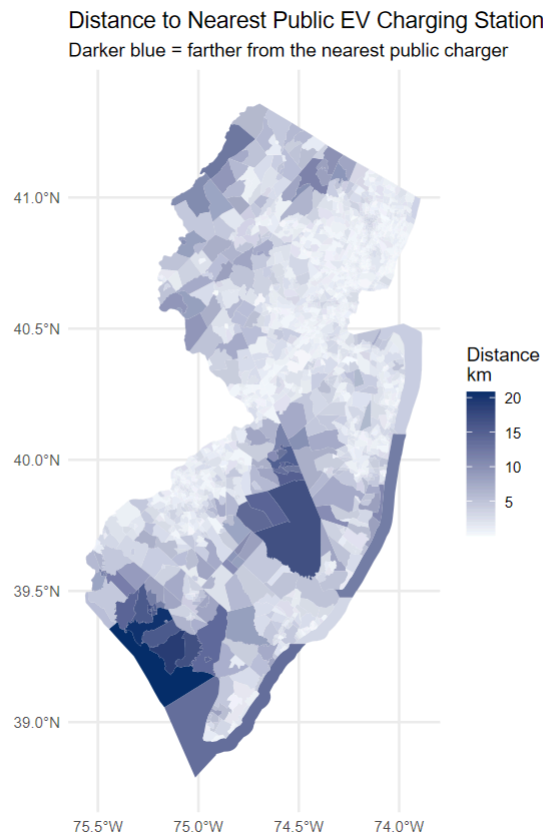
tract_points <- st_point_on_surface(tract_geoms_5070)
nearest_id <- st_nearest_feature(tract_points, ev_geoms_5070)

final_data$dist_nearest_charger_m <- as.numeric(
  st_distance(tract_points, ev_geoms_5070[nearest_id], by_element = TRUE)
)

final_data$dist_nearest_charger_km <- final_data$dist_nearest_charger_m / 1000
```

In [543...

```
ggplot(final_data) +  
  geom_sf(aes(fill = dist_nearest_charger_km), color = NA) +  
  scale_fill_gradient(  
    low = "#F7FBFF",  
    high = "#08306B",  
    na.value = "gray85",  
    name = "Distance\nkm"  
  ) +  
  labs(  
    title = "Distance to Nearest Public EV Charging Station",  
    subtitle = "Darker blue = farther from the nearest public charger"  
  ) +  
  theme_minimal()
```



10. Spatial autocorrelation

Moran's I tests whether nearby tracts tend to have similar charging rates. A significant positive result means charging availability is spatially clustered rather than randomly scattered.

```
In [544... library(spdep)

moran_data <- final_data %>%
  filter(!is.na(stations_per_10k_clean)) %>%
  st_transform(5070)

neighbors <- poly2nb(moran_data, queen = TRUE)

weights <- nb2listw(
  neighbors,
  style = "W",
  zero.policy = TRUE
)

moran_result <- moran.test(
  moran_data$stations_per_10k_clean,
  weights,
  zero.policy = TRUE
)

moran_result
```

Moran I test under randomisation

data: moran_data\$stations_per_10k_clean

weights: weights

Moran I statistic standard deviate = 7.2733, p-value = 1.754e-13

alternative hypothesis: greater

sample estimates:

Moran I statistic	Expectation	Variance
0.0907234270	-0.0004621072	0.0001571778

```
In [ ]: final_data <- final_data %>%
  mutate(
    land_area_sq_km = ALAND / 1e6,
    population_density = ifelse(land_area_sq_km > 0, total_population / land_area_sq_km, NA_real_)
  )
```

11. Regression setup and baseline model

Regression comes after EDA and mapping. The negative binomial model treats station count as the outcome and uses tract population as an offset, so it estimates station availability relative to population size.

We use a count model because station counts are nonnegative integers and the EDA shows many zero-station tracts with a long right tail. Coefficients should be read as associations, not causal effects.

```
In [546... model_data <- final_data %>%
  st_drop_geometry() %>%
  filter(
    !is.na(stations_total),
    !is.na(median_income),
    !is.na(poverty_rate),
    !is.na(pct_no_vehicle),
    !is.na(pct_owner_occupied),
    is.finite(population_density),
    total_population >= 500
  ) %>%
  mutate(
    median_income_10k = median_income / 10000,
    log_population = log(total_population),
    log_pop_density = log1p(population_density)
  )

summary(model_data$median_income)
nrow(model_data)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
15943	75625	103487	112014	139922	250001

Fit the baseline negative binomial model for station counts.

```
In [547... library(MASS)

nb_model <- glm.nb(
  stations_total ~ median_income_10k + poverty_rate + pct_no_vehicle + pct_owner_occupied + offset(log_population),
  data = model_data
)

summary(nb_model)
```

Call:

```
glm.nb(formula = stations_total ~ median_income_10k + poverty_rate +
  pct_no_vehicle + pct_owner_occupied + offset(log_population),
  data = model_data, init.theta = 0.2999901432, link = log)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-8.321929	0.271418	-30.661	< 2e-16 ***
median_income_10k	0.037552	0.012791	2.936	0.00333 **
poverty_rate	-0.008123	0.007929	-1.024	0.30561
pct_no_vehicle	0.001434	0.005685	0.252	0.80086
pct_owner_occupied	-0.009163	0.002853	-3.212	0.00132 **

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for Negative Binomial(0.3) family taken to be 1)

Null deviance: 1573.3 on 2148 degrees of freedom
Residual deviance: 1552.7 on 2144 degrees of freedom
AIC: 5030

Number of Fisher Scoring iterations: 1

Theta: 0.3000
Std. Err.: 0.0190

2 x log-likelihood: -5018.0170

12. Baseline model diagnostics

After fitting the baseline model, we inspect residuals. Residual maps and residual outlier tables show where the model overpredicts or underpredicts station counts.

The residual Moran test checks whether unexplained model error is still spatially clustered. If it is, that suggests the model is missing geographic structure.

```
In [548...] model_data_resid <- model_data %>%
  mutate(
    fitted_stations = fitted(nb_model),
    pearson_resid = residuals(nb_model, type = "pearson"),
    deviance_resid = residuals(nb_model, type = "deviance"),
    raw_resid = stations_total - fitted_stations
  )
```

```
In [549...] resid_map_data <- final_data %>%
  left_join(
    model_data_resid %>%
      dplyr::select(GEOID, fitted_stations, raw_resid, pearson_resid, deviance_resid),
    by = "GEOID"
  )
```

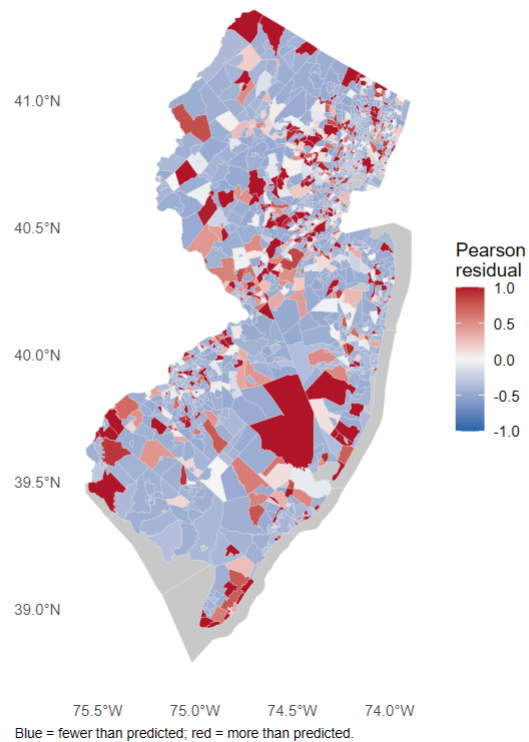
```
In [557...] ggplot(resid_map_data) +
  geom_sf(aes(fill = pearson_resid), color = "gray90", linewidth = 0.05) +
  scale_fill_gradient2(
    low = "#2166AC",
    mid = "#F7F7F7",
    high = "#B2182B",
    midpoint = 0,
    limits = c(-1, 1),
    oob = scales::squish,
    na.value = "gray80",
    name = "Pearson\nresidual"
  ) +
  labs(
    title = "Model Residuals: Public EV Charging Stations",
```

```

    subtitle = "Scale capped at  $\pm 1$ ",
    caption = "Blue = fewer than predicted; red = more than predicted."
) +
theme_minimal() +
theme(
  panel.grid = element_blank(),
  plot.caption.position = "plot",
  plot.caption = element_text(hjust = 0, size = 8)
)

```

Model Residuals: Public EV Charging Stations
Scale capped at ± 1



In [551...

```

summary(resid_map_data$pearson_resid)
range(resid_map_data$pearson_resid, na.rm = TRUE)

```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
-0.51941	-0.47352	-0.44292	0.00804	0.03971	21.14092	32

-0.519411685718238 · 21.1409150534634

```
In [552... model_data_resid %>%  
  arrange(desc(pearson_resid)) %>%  
  dplyr::select(  
    GEOID,  
    stations_total,  
    fitted_stations,  
    raw_resid,  
    pearson_resid,  
    median_income,  
    poverty_rate,  
    pct_no_vehicle,  
    pct_owner_occupied  
  ) %>%  
  head(10)
```


A data.frame: 10 × 9

	GEOID	stations_total	fitted_stations	raw_resid	pearson_resid	median_income	poverty_rate	pct_no_vehicle	pct_owner_occupied
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	34001002400	23	0.4530950	22.546905	21.140915	26427	47.3227945	59.735974	4.180418
2	34003032103	28	0.7947435	27.205256	15.974916	105184	1.5255361	5.736138	78.087954
3	34013006400	7	0.2125679	6.787432	11.262600	80345	24.1430700	11.253561	62.535613
4	34033022201	7	0.2640714	6.735929	9.559321	80000	7.9505300	11.285714	77.285714
5	34003042500	16	0.8247724	15.175228	8.629611	101068	4.2836744	3.173531	63.200540
6	34023001417	10	0.4882122	9.511788	8.398320	222083	0.9788769	6.485671	94.419306
7	34029738002	5	0.2153161	4.784684	7.867482	139464	3.2151690	5.962521	90.459966
8	34029728003	6	0.2890880	5.710912	7.579796	45694	41.9261822	29.617117	27.364865
9	34009022000	8	0.4369560	7.563044	7.299840	62576	10.3057758	5.980529	64.534075
10	34025808501	11	0.7548961	10.245104	6.288158	155758	4.5381310	2.601734	80.053369



In [553...

```
model_data_resid %>%  
  arrange(pearson_resid) %>%  
  dplyr::select(  
    GEOID,  
    stations_total,  
    fitted_stations,  
    raw_resid,  
    pearson_resid,  
    median_income,  
    poverty_rate,  
    pct_no_vehicle,  
    pct_owner_occupied  
  ) %>%  
  head(10)
```

A data.frame: 10 × 9

	GEOID	stations_total	fitted_stations	raw_resid	pearson_resid	median_income	poverty_rate	pct_no_vehicle	pct_owner_occupied
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	34029715701	0	2.679784	-2.679784	-0.5194117	80547	23.33260917	4.087531	41.32948
2	34039036800	0	2.405089	-2.405089	-0.5164510	250001	1.57748612	7.036092	78.92818
3	34003055100	0	2.202956	-2.202956	-0.5138433	247292	2.83667203	0.000000	68.83969
4	34013020500	0	2.182758	-2.182758	-0.5135586	250001	0.63594912	1.031322	93.92666
5	34023003601	0	2.087031	-2.087031	-0.5121413	115798	3.86955464	5.714286	40.16097
6	34021004501	0	2.086839	-2.086839	-0.5121384	197434	6.45994832	13.656388	49.92658
7	34023001504	0	2.062468	-2.062468	-0.5117585	142517	6.17283951	6.990881	32.52280
8	34039036400	0	2.049298	-2.049298	-0.5115498	250001	0.06625149	1.328502	90.74074
9	34029715600	0	1.959540	-1.959540	-0.5100603	75893	16.38750122	9.779688	47.07147
10	34039038601	0	1.931832	-1.931832	-0.5095753	144716	1.89760571	2.321020	58.22164



In [554...

```
resid_moran_data <- resid_map_data %>%  
  filter(!is.na(pearson_resid)) %>%  
  st_transform(5070)  
  
resid_neighbors <- poly2nb(resid_moran_data, queen = TRUE)  
  
resid_weights <- nb2listw(  
  resid_neighbors,  
  style = "W",  
  zero.policy = TRUE  
)  
  
resid_moran <- moran.test(  
  resid_moran_data$pearson_resid,  
  resid_weights,  
  zero.policy = TRUE
```

```
)
```

```
resid_moran
```

```
      Moran I test under randomisation
```

```
data: resid_moran_data$pearson_resid
```

```
weights: resid_weights
```

```
Moran I statistic standard deviate = 7.5964, p-value = 1.523e-14
```

```
alternative hypothesis: greater
```

```
sample estimates:
```

Moran I statistic	Expectation	Variance
0.0951816591	-0.0004655493	0.0001585377

13. Model refinement: population density

Population density is added as a simple geographic control because station placement is often tied to urban form, commercial activity, and travel demand. This is a refinement after the baseline model

adds population density.

In [555...

```
nb_model_density <- glm.nb(  
  stations_total ~ median_income_10k + poverty_rate + pct_no_vehicle +  
    pct_owner_occupied + log_pop_density + offset(log_population),  
  data = model_data  
)  
  
summary(nb_model_density)
```

```
Call:
glm.nb(formula = stations_total ~ median_income_10k + poverty_rate +
      pct_no_vehicle + pct_owner_occupied + log_pop_density + offset(log_population),
      data = model_data, init.theta = 0.3338794067, link = log)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-4.756224	0.453288	-10.493	< 2e-16 ***
median_income_10k	0.050560	0.012382	4.083	4.44e-05 ***
poverty_rate	-0.014622	0.007871	-1.858	0.0632 .
pct_no_vehicle	0.011564	0.005740	2.015	0.0439 *
pct_owner_occupied	-0.021257	0.003039	-6.994	2.67e-12 ***
log_pop_density	-0.419317	0.043286	-9.687	< 2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for Negative Binomial(0.3339) family taken to be 1)

Null deviance: 1666.1 on 2148 degrees of freedom
Residual deviance: 1559.7 on 2143 degrees of freedom
AIC: 4950.9

Number of Fisher Scoring iterations: 1

Theta: 0.3339
Std. Err.: 0.0217

2 x log-likelihood: -4936.9330